

Технічна специфікація (Engineering Specification): Архітектура AI-системи генерації DRAKON-схем

У сучасній індустрії розробки програмного забезпечення інтеграція великих мовних моделей (LLM) у процеси візуалізації архітектури стикається з фундаментальними топологічними обмеженнями. Класичні інструменти генерації діаграм (такі як Mermaid.js, PlantUML чи Graphviz) базуються на алгоритмах гравітаційного макетування або методах балансування маси, які неминуче призводять до візуального хаосу при обробці складних алгоритмічних структур.¹ Для вирішення цієї проблеми розроблено візуальну мову DRAKON, що початково створювалася для аерокосмічної програми «Буран» з метою абсолютної мінімізації когнітивного навантаження інженерів шляхом впровадження жорсткої ортогональної геометрії та заборони перетинів ліній.¹

Цей документ становить вичерпну інженерну специфікацію для створення автоматизованого конвеєра "Code-to-Diagram", що перетворює вихідний код або неструктурований текст на суворо стандартизовані DRAKON-схеми.¹ Архітектурною парадигмою системи є **Hybrid Topological Semantic Extraction** (Гібридна топологічно-семантична екстракція), яка чітко розмежовує семантичний аналіз (делегований LLM) та геометричне макетування (делеговане детермінованим алгоритмам на кшталт drakonwidget.js або ELK.js).¹ Такий підхід повністю нівелює ризик "просторових галюцинацій" трансформерних моделей, оскільки штучний інтелект оперує виключно планарними списками суміжності (adjacency lists) без розуміння двовимірних координат.¹

Блок А — Доменна модель

Основою архітектури є формалізація доменної моделі DRAKON, оптимізованої для машинного навчання та статичного аналізу коду. Модель описує структури даних, що відповідають за візуалізацію, абстрагуючись від фізичного рендерингу пікселів на екрані.

1. Canonical Entities (Канонічні сутності системи)

На відміну від традиційних систем побудови графів (наприклад, React Flow), які оперують двома окремими масивами — вузлами (nodes) та зв'язками (edges), доменна модель DRAKON використовує парадигму плоского словника ідентифікаторів (flat JSON-dictionary).¹ Ця модель складається з наступних канонічних сутностей:

- **Diagram (Діаграма):** Кореневий контейнер, що капсулює весь алгоритмічний простір. Він містить глобальні метадані, такі як назва алгоритму, конфігурації стилів, параметри доступу та центральний об'єкт items, який виступає словником усіх елементів графа.¹

- **Item (Ікона / Вузол):** Базова неподільна алгоритмічна одиниця. Кожен об'єкт Item ідентифікується унікальним строковим ключем і містить власну семантичну роль (тип фігури), текстовий контент (бізнес-логіку або код) та топологічні вказівники.¹
- **Implicit Edge (Неявний зв'язок):** Сутність ліній у моделі DRAKON не існує як окремий об'єкт. Зв'язки реалізуються виключно через топологічні вказівники one (наступний крок по основному вектору) та two (альтернативний або помилковий крок) всередині самого об'єкта Item.¹ Це дозволяє виконувати пошук сусідів за час $O(1)$ і радикально зменшує ймовірність структурних помилок під час генерації LLM.
- **Primitive (Примітив):** Макроструктура нижнього рівня, що становить собою лінійний алгоритм з єдиною точкою входу (Start) та єдиною точкою виходу (End), який розгортається навколо однієї центральної вертикальної осі виконання.¹
- **Silhouette (Силует):** Макроструктура верхнього рівня, призначена для декомпозиції надзвичайно складних алгоритмів. Силует розрізає монолітний граф на семантично незалежні блоки ("гілки"), які вишиковуються паралельно одна одній на єдиному візуальному полотні.¹
- **Branch (Гілка):** Окрема колонка дій всередині Силуету. Вона обов'язково починається з ікони-заголовка, має власний ізольований шампур і завершується навігаційною командою переходу до іншої гілки алгоритму.¹

2. Structural Constraints (Структурні обмеження)

Топологія DRAKON накладає суворі математичні обмеження на граф потоку управління (Control Flow Graph), порушення яких робить схему принципово неможливою для компіляції або відмальовування.¹

1. **Zero-Intersection Policy (Політика нульових перетинів):** У коректній схемі DRAKON абсолютно неможливе існування ліній зв'язку, що перетинають одна одну. Будь-який перетин вимагає рефакторингу логіки шляхом розширення площини малювання або застосування декомпозиції (переходу від Примітиву до Силуету).¹ Це фундаментальне правило ліквідує необхідність витратити когнітивні ресурси на розплутування візуального павутиння.
2. **The Single Terminal Constraint (Один термінал):** Будь-яка алгоритмічна сутність повинна завершуватися виключно одним вузлом типу End.¹ Множинні точки виходу (наприклад, декілька операторів return у різних гілках) повинні бути топологічно зведені маршрутизатором до одного спільного візуального завершення.
3. **Vector Constraints for Bifurcations:** Умовні вузли розгалуження (типу Question або If) не мають права скеровувати потік управління ліворуч.¹ Виходи дозволені виключно вниз (що зазвичай відповідає істинній умові) та вправо (що відповідає хибній умові або винятку).¹ Спроба створити лівобічний вихід розцінюється як критичне порушення інваріанту.
4. **Anti-Twin Error (Заборона прямих горизонтальних зв'язків):** У структурі Силуету паралельні гілки не можуть з'єднуватися між собою візуальними лініями напряму. Передача управління можлива лише через вузол навігації Address, який вказує на

ім'я цільової гілки.¹

5. **Encapsulation of Jumps:** Забороняються стрибки всередину абстракцій. Вказівник переходу не може бути направлений у середину іншої алгоритмічної гілки — він зобов'язаний вказувати виключно на стартовий заголовок гілки (на її точку входу).¹

3. Layout Constraints (Обмеження макетування)

Геометричне позиціонування елементів не є завданням LLM, проте алгоритм рушія рендерингу (drakonwidget.js або ELK.js) керується наступними принципами просторового розрахунку¹:

- **Manhattan Routing (Ортогональність):** Усі лінії зв'язку мають будуватися виключно з використанням вертикальних та горизонтальних сегментів, поєднаних під кутом 90 градусів.¹ Використання діагональних, хвилястих або вигнутих сплайнів (як у D3.js) категорично заборонено, оскільки людське око витрачає мікрозусилля на відстеження кривих.¹
- **Vertical Gravity:** Всі дієві вузли (іконки команд, умов, викликів) повинні розташовуватися строго на вертикальних осях, формуючи стовпці. Горизонтальні лінії слугують виключно для маршрутизації потоку після розгалужень та повернення до основної осі.¹
- **Alignment of Symmetrical Paths:** Рушій макетування повинен забезпечувати симетричне вирівнювання іконок, що належать до паралельних шляхів виконання після розгалуження, щоб створювати збалансовану візуальну масу графа.

4. Readability Constraints (Обмеження читабельності)

Когнітивна ергономіка DRAKON формалізується через набір евристик, які не впливають на виконання коду, але є критичними для експертного сприйняття.¹

- **The Skewer Rule (Правило Шампура):** Найбільш бажаний і найчастіше використовуваний шлях виконання програми ("happy path") повинен формувати ідеальну, пряму вертикальну вісь ("шампур").¹ Архітектор, що переглядає код, не повинен здійснювати горизонтальних пошукових рухів для розуміння головної мети функції.¹ Злам шампура (його зміщення) є маркером неякісного рефакторингу логіки.
- **The Rightward Degradation Heuristic (Чим правіше, тим гірше):** Альтернативні маршрути повинні ранжуватися по осі X.¹ Що далі сценарій відхиляється вправо від основного шампура, то гіршим або більш винятковим він вважається. Відповідно, блоки обробки фатальних помилок, винятки catch та малоімовірні крайові випадки (edge cases) примусово виштовхуються на крайню праву периферію діаграми.¹ Око інженера автоматично ідентифікує праву зону як зону ризику.
- **Chronological Left-to-Right Ordering:** Гілки макрошаблону Силует повинні просторово впорядковуватися зліва направо у строгому хронологічному порядку їх виконання.¹ Хоча логіка дозволяє переходи в будь-якому напрямку, правильне

просторове сортування забезпечує узгодженість часового виміру з візуальним простором.

Блок В — Rules Catalog (Каталог правил)

Успішна конвертація вихідного коду або тексту в діаграму DRAKON передбачає розв'язання задачі задоволення обмежень (Constraint Satisfaction Problem).¹ Наведена нижче система класифікації поділяє правила на жорсткі (інваріанти), м'які (ергономічні), такі, що піддаються автоматичному виправленню (autofix), та ті, що вимагають втручання людини. Ці правила є контрактом для мікросервісу Graph Validator.

Правило	Тип	Що ламається при порушенні	Autofix можливий?	Validator Check (Алгоритм перевірки)	LLM Instruction (Вказівка моделі)
Відсутність перетинів ліній зв'язку. ¹	Hard	Візуальний рендеринг, граф стає топологічно невалідним (Spaghetti code).	Так	Делегується рушію рендерингу (drakonwid.get.js), який розширює площину для уникання перетинів.	Тобі ЗАБОРОНЕНО генерувати координати (X, Y). Фокусуйся лише на вказівниках переходів (one/two).
Один вузол End на весь алгоритм. ¹	Hard	Неможливість коректної компіляції AST; руйнується	Так	Count(nodes where type == "end") == 1. Якщо > 1, валідатор	Завершуй усі кінцеві шляхи виконання посиланням на єдиний

		закриваючий scope (наприклад, return).		програмно зливає всі термінальні one вказівники на єдиний згенерований End вузол.	ідентифікатор вузла end.
Виходи з вузла question (умови) виключно вниз і вправо.¹	Hard	Руйнується гравітаційний потік читання, лінії згинаються непередбачувано.	Так	Перевірка схеми JSON: вузол question має містити виключно атрибути переходу one та two. Будь-які спроби створити третій перехід блокуються.	Скеровуй "True" (або успішний) шлях в атрибут one, а "False" (або виняток) — в атрибут two.
Нерозривність "Happy Path" на головному шампурі.¹	Soft	Когнітивна читабельність ("глянув-і-зрозумів"); мозок витрачає час на пошук основної логіки.	? Неоднозначно	Вимагає семантичного аналізу коду, що важко детермінувати жорстким кодом. Частково	Аналізуй бізнес-логіку і завжди призначай найбільш оптимальний сценарій як пряме продовже

				перевіряється довжиною ланцюга one.	ння (one).
Чим правіше, тим гірше (Rightward Degradation).¹	Soft	Когнітивний дисонанс: позитивні рішення знаходяться там, де інженер очікує побачити помилки.	Ні	Потребує оцінки семантики "успішності" блоку коду, що є зоною відповідальності AI Critic Model або Human Review.	Усі блоки обробки помилок, catch винятки та переривання обов'язково відкидай у бічні гілки (через two).
Заборона стрибків у середину гілки Силуету.¹	Hard	Еквівалент шкідливого goto. Руйнується модульність ("чорний ящик" гілки).	Так	Вузли типу address повинні містити посилання one, що вказує виключно на ідентифікатори вузлів типу branch.	Навігація в Силуеті (через address) можлива лише на заголовки гілок, ніколи на кроки в їх середині.
Відсутність незв'язаних вузлів ("сиріт").¹	Hard	Наявність логіки, яка ніколи не буде виконана;	Так	Обхід графа алгоритмом Depth-First	Не генеруй ізольовані ідентифікатори.

		засмічення пам'яті та екрану.		Search (DFS) від стартового вузла. Всі невідвідані вузли автоматично видаляються валідатором.	Кожен вузол має бути досяжним з початкового блоку.
Замикання множинного вибору пустим case.¹	Soft	Можливість втрати непередбачених умов у структурі Switch/Case; логічні діри.	Так	Перевірка: чи останній case у ланцюзі розгалуження має порожній content. Якщо ні — валідатор ін'єктує його.	Для блоків вибору завжди додавай порожній case (default) як останній варіант для всіх інших випадків.

Блок С — Icon Taxonomy (Таксономія іконок)

Класичний стандарт DRAGON налічує 27 базових іконок та 21 макроікону.¹ Для забезпечення інтеграції з drakonwidget.js ця множина формалізована у строгу таксономію підтримуваних типів (type property), яка визначає візуальну фігуру та допустимі зв'язки.¹

Ця таблиця виступає специфікацією для генерації Enum-значень у JSON Schema.

icon (type)	group	призначення та	допустима позиція	hard constraint	autofix можливи
-------------	-------	----------------	-------------------	-----------------	-----------------

		семантика		s (обмеження)	й
header	Structure	Заголовок алгоритму / Точка входу (Begin). Визначає назву функції.	Зверху найлівішого шампура (для примітивів).	Дозволено лише один на весь алгоритм (якщо це не Силует). Має один вихід.	? Так (ін'єкція дефолтного імені).
end	Structure	Логічне та фізичне завершення виконання алгоритму.	Внизу крайнього правого шампура алгоритму.	Строго один на весь граф. Не має вихідних зв'язків.	Так (злиття всіх відкритих one у єдиний ID).
branch	Structure	Заголовок паралельної гілки макрощаблону Силует.	Зверху власної гілки (колонки).	Повинен мати унікальне ім'я та порядковий branchId. ³	Так (автоматична ітеративна нумерація branchId).
address	Structure	Точка навігації (передачі управління) до іншої гілки Силуету.	Внизу будь-якої гілки (крім останньої правої).	Перехід дозволено лише на ім'я існуючого branch. Заборонено вказувати в	Так (мапінг вказівника на найближчий існуючий ID гілки).

				середину логіки. ⁴	
action	Behavioral	Процес (прямокутник). Виконання імперативних команд, обчислень, викликів. ⁸	Виключно на вертикальній осі (шампурі).	Повинен мати рівно один вхід і один вихід (атрибут one).	Так (усунення або злиття пустих action блоків).
question	Flow	Логічне розгалуження (If / Condition). Фігура — ромб. ¹	На вертикальній осі (шампурі).	Має два виходи: one (вниз) і two (вправо). Вимагає атрибут flag1 для орієнтації міток Yes/No. ³	Так (програмне перепризначення вказівника two на пусту гілку).
select	Flow	Точка розгалуження множинного вибору (Switch).	На вертикальній осі.	Мусить з'єднуватися горизонтальною лінією із серією вузлів типу case. ¹	? Ні (порушення бізнес-логіки вимагає розробника).
case	Flow	Варіант вибору або умова для блоку	Розміщується поруч по горизонта	Працює виключно у зв'язці з батьківськ	Так (автоматичне додавання

		select. ⁸	лі, під вузлом select.	им select. Крайній правий case діє як default. ¹	крайнього пустого case).
foreach	Flow	Макроікон а для ітерації (For loop) по колекції або діапазону. ¹	На вертикаль ній осі.	Потребує парних ікон (початок і кінець циклу), між якими розміщуєт ься тіло. ¹	Так (генерація парного закриваюч ого вузла рушієм).
insertion	Sub-proce ss	Виклик підпрогра ми (макрос). Потік чекає на поверненн я результат у. ¹	На вертикаль ній осі.	Представ ляє "чорний ящик", логіка якого визначена в іншій схемі. ⁸	Ні.
input	I/O	Отриманн я даних або подій із зовнішньо го світу. ⁸	На вертикаль ній осі.	Традиційн о двокамер на ікона (дія + об'єкт), але підтримує ться і спрощена (однокаме рна). ¹	Так (автомати чне розбиття сирого тексту на дві секції).
output	I/O	Відправле ння даних,	На вертикаль	Аналогічн о input,	Так (парсинг

		команд або подій назовні. ⁸	ній осі.	розділення дієслова та іменника.	та розділення тексту).
shelf	External	"Полиця". Багатоцільова ікона для присвоєння, наказів акторам або повідомлень. ¹	На вертикальній осі.	Вимагає використання поля secondary для нижньої секції полиці. ³	Так (форматування контенту між content та secondary).
process	Concurrenсу	Запуск асинхронного фонового процесу. ⁸	На вертикальній осі.	Допускає команди Start, Pause, Continue, Stop у верхній секції. ⁸	Ні.
timer	Time	Ініціалізація таймера/лічильника систем реального часу. ⁸	На вертикальній осі.	Працює у зв'язці з блоками duration. ⁸	Ні.
duration	Time	Затримка виконання (delay) або обмеження тривалості дії. ¹	Прикріпле на ліворуч до іншої ікони дії.	Має зв'язуватися з основним вузлом через атрибут side. ³	Так (авто-кріплення side до найближчого Action).

Блок D — Intermediate Representation (IR) Specification

Центральною проблемою перетворення неструктурованого коду або тексту у візуальні діаграми за допомогою штучного інтелекту є висока ймовірність "просторових галюцинацій". LLM (такі як GPT-4 чи Claude) не мають внутрішнього двовимірного рендеру, що призводить до накладання вузлів та плутанини у лініях, якщо вимагати від них генерувати SVG або абсолютні координати (X, Y).¹

1. Canonical Intermediate Representation

Для забезпечення математичної точності та ергономіки DRAKON, проміжним представленням (Canonical IR) визначено **Flat JSON Adjacency List** (плоский JSON-словник ідентифікаторів).¹

Замість генерації глибоко вкладених синтаксичних дерев (AST), де випадково пропущена закриваюча дужка руйнує весь граф, LLM формує плоский масив об'єктів. Кожен об'єкт має унікальний строковий ключ-ідентифікатор, а зв'язки між ними визначаються простими строковими вказівниками. Це дозволяє моделі генерувати алгоритм ітеративно, імітуючи роботу вказівників у пам'яті (pointers).

2. Mapping IR → DRAKON Diagram Model

Трансформація IR у фінальну відмальовану схему здійснюється рушієм (наприклад, `drakonwidget.js`) за наступною логікою відображення:

- **Створення Шампура:** Рушій знаходить стартовий вузол (де `type: "header"`) і слідує за його атрибутом `one`. Всі вузли, з'єднані ланцюгом `one`, вишиковуються на єдиній прямій вертикальній осі (ідеальний "Happy Path").
- **Створення Розгалужень:** Коли рушій зустрічає вузол `question` або `select`, атрибут `two` сигналізує про створення нової паралельної вертикальної осі **праворуч** від поточної.
- **Злиття (Common Fate Merge):** Якщо рушій виявляє, що вказівник `one` з основної гілки та вказівник `one` з побічної гілки вказують на однаковий цільовий ідентифікатор, він математично розраховує горизонтальну лінію злиття (`merge point`), автоматично усуваючи перетини за допомогою Манхеттенської маршрутизації.¹

3. Розподіл відповідальності (Responsibility Boundary)

Архітектура суворо розмежовує те, що генерує ШІ, і те, що обчислює детермінований код.

- **Що генерує LLM (Semantic Marking):** Унікальні ID вузлів, типи іконок (`type`), витягнений з AST текстовий контент бізнес-логіки (`content`), та логічні вказівники на

наступні операції (one, two, side).¹

- **Що генерує Deterministic Code (Після LLM):** Розрахунок фізичної геометрії, обчислення ширини/висоти блоків залежно від обсягу тексту, обчислення координат X та Y, трасування ортогональних ліній без перетинів та призначення розмірів Canvas.¹

Блок Е — JSON Schema (Draft)

Наведена нижче JSON Schema описує контракт між LLM, валідатором та фронтенд-рушієм drakonwidget.js. Схема призначена для інтеграції в Model Context Protocol (MCP) сервери або як response_format для викликів OpenAI/Anthropic API.¹ Вона гарантує цілісність полів та базову перевірку типів.³

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "DrakonDiagramIR",
  "description": "Flat JSON adjacency list representing a DRAKON algorithmic flow.",
  "type": "object",
  "required": ["name", "items"],
  "properties": {
    "name": {
      "type": "string",
      "description": "The name of the algorithm or business process."
    },
    "access": {
      "type": "string",
      "enum": ["read", "write"],
      "default": "read",
      "description": "Access level for the diagram editor."
    },
    "params": {
      "type": "string",
      "description": "Input parameters for the function/algorithm."
    },
    "items": {
      "type": "object",
      "description": "Dictionary of diagram nodes. Keys are unique string IDs (e.g., 'node_1', 'step_auth').",
    }
  }
}
```

```

    "patternProperties": {
      "^[a-zA-Z0-9_-]+$": {
        "$ref": "#/$defs/dragonItem"
      }
    },
    "additionalProperties": false,
    "minProperties": 2
  },
  "$defs": {
    "dragonItem": {
      "type": "object",
      "required": ["type", "content"],
      "properties": {
        "type": {
          "type": "string",
          "enum": [
            "header", "end", "action", "question", "select", "case",
            "foreach", "insertion", "input", "output", "shelf",
            "process", "timer", "duration", "address", "branch"
          ],
        },
        "description": "The structural role and visual shape of the icon."
      },
      "content": {
        "type": "string",
        "description": "Main logic payload, pseudocode, or natural text."
      },
      "secondary": {
        "type": "string",
        "description": "Secondary content for dual-chamber icons like 'shelf'."
      },
      "one": {
        "type": "string",
        "description": "Pointer to the ID of the next item on the main vertical path (Happy Path)."
      },
      "two": {
        "type": "string",
        "description": "Pointer to the ID of the alternative, error, or 'No' path (branched right)."
      },
      "side": {
        "type": "string",
        "description": "Pointer to the ID of a duration item attached to the left."
      },
    },
  },

```

```

    "flag1": {
      "type": "integer",
      "description": "Orientation of the Yes/No labels for Question icons (typically 0 or 1).",
    },
    "branchId": {
      "type": "integer",
      "description": "Sequence ordinal for Silhouette branches to enforce chronological order.",
    },
    "style": {
      "type": "string",
      "description": "Stringified JSON object containing localized visual overrides (e.g., iconBack, color).",
    },
  },
  "additionalProperties": false
}
}
}

```

Приклад валідного IR (Valid IR)

Цей приклад демонструє базовий алгоритм валідації доступу, закодований як "Примітив" з розгалуженням та злиттям.

JSON

```

{
  "name": "UserAuthentication",
  "access": "read",
  "items": {
    "node_start": {
      "type": "header",
      "content": "Check Auth",
      "one": "node_get_token"
    },
    "node_get_token": {
      "type": "action",
      "content": "Extract JWT token from headers",
      "one": "node_is_valid"
    },
    "node_is_valid": {

```

```

    "type": "question",
    "content": "Is token signature valid?",
    "one": "node_grant_access",
    "two": "node_deny_access",
    "flag1": 0
  },
  "node_grant_access": {
    "type": "action",
    "content": "Set User Context\nReturn HTTP 200",
    "one": "node_end"
  },
  "node_deny_access": {
    "type": "action",
    "content": "Log Security Event\nReturn HTTP 403",
    "one": "node_end"
  },
  "node_end": {
    "type": "end",
    "content": "End"
  }
}

```

Приклад невалідного IR (Invalid IR)

Наступний приклад демонструє класичну помилку генерації LLM, яка вимагає втручання валідатора.

JSON

```

{
  "name": "BrokenProcess",
  "items": {
    "step_1": {
      "type": "question",
      "content": "Check database status",
      "one": "step_2",
      "two": "step_error"
    },
    "step_2": {

```

```

    "type": "action",
    "content": "Fetch records",
    "one": "step_1"
  }
}
}

```

Пояснення порушень:

1. **Dangling Pointer (Вказівник у нікуди):** Вузол step_1 посилається в атрибуті two на ідентифікатор step_error, який відсутній у словнику items. Рендеринг впаде.
2. **Missing Header and End (Відсутні термінали):** Алгоритм не має точок входу та виходу.
3. **Invalid Loop (Хибний цикл):** Вузол step_2 посилається назад на step_1 створюючи безкінечний цикл без правильного оформлення макроіконою foreach.
Як виправити (Autofix Module): Модуль валідації автоматично ін'єктує вузол header, згенерує пустий вузол-заглушку step_error (щоб задовольнити вказівник), і перенаправить некоректний цикл на ін'єктований вузол end.

Блок F — LLM Instruction Contract

Успішна робота гібридної архітектури вимагає бездоганного System Prompt (контракту) для налаштування контекстного вікна мовної моделі. Цей контракт визначає, як LLM повинна "мислити" простором та логікою під час перекладу коду в граф.¹

1. Обов'язкові директиви (Mandates)

- **Твоє завдання — Семантична Екстракція:** Ти зобов'язана аналізувати вихідний код, витягувати з нього граф потоку управління (Control Flow Graph) і класифікувати логічні ролі кожного блоку.
- **Створи Adjacency List:** Весь результат має бути згенерований виключно як плоский словник ідентифікаторів у форматі JSON.
- **Забезпечуй "Happy Path":** Головний, найбільш ймовірний сценарій успішного виконання алгоритму ти зобов'язана завжди спрямовувати через топологічний атрибут one (вниз).¹
- **Застосовуй згортання (Simplification):** Якщо у коді наявні 3-4 послідовні лінійні операції (наприклад, декларації змінних, базові присвоєння), які не містять умов чи циклів, ти повинна згорнути їх у єдиний об'єднаний вузол action, розділивши рядки символом \n. Це критично для зменшення візуального шуму.¹

2. Категоричні заборони (Prohibitions)

- **Жодної геометрії:** Тобі категорично заборонено вигадувати, припускати або

обчислювати X/Y координати, розміри пікселів, відступи чи кути нахилу ліній.¹ Твоя зона відповідальності — виключно топологія.

- **Жодних лівобічних виходів:** При генерації вузла question забороняється формувати логіку так, щоб потік потребував руху вліво.¹ Використовуй лише атрибути one (вниз) та two (вправо).³
- **Стережись множинних фіналів:** Заборонено створювати більше ніж один вузол з type: "end". Усі переривання виконання (навіть return або throw Error) повинні топологічно лінкуватися (через one) на єдиний термінальний ідентифікатор графа.¹

3. Contract Matrix (Матриця поведінки LLM)

Ситуація у вхідному коді / тексті	Що LLM має зробити (Expected Action)	Що LLM НЕ має робити (Anti-pattern)	Як позначати невизначеність
Блоки try/catch, валідація, обробка помилок.	Створити вузол обробки та призначити його ID у поле two батьківського вузла question (зсув вправо).	Залишати обробку помилки у полі one (на головному шампурі).	Додати коментар: /* Potential Error Path */ всередині content.
Множинне розгалуження (switch/case).	Створити вузол select, який посилається на серію вузлів case. Останній case зробити порожнім (дефолтним).	Конвертувати switch у довгий ланцюг вкладених question (if-else-if-else).	-
Динамічна типізація (неоднозначний тип змінної).	Створити вузол action з оригінальним синтаксисом змінної.	Припускати або галуцинувати типи даних (наприклад, додавати int чи string), яких не було	Зазначити у content: varName =... /* Type unknown */.

		в джерелі.	
Стрибки (goto) у legacy коді.	Трансформувати логіку в декомпозований "Силует". Вузли address повинні вказувати на заголовок відповідної гілки.	Лінкувати address безпосередньо всередину гілки (на випадковий action).	Якщо точка переходу втрачена, лінкувати на end і додати префікс [?] у текст.

4. Обмеження складності (Complexity Boundaries)

Для запобігання перетворенню діаграми на хаотичну структуру ("cobweb"), LLM повинна дотримуватися таких лімітів ¹:

- **Глибина вкладеності (Rightward Drift):** Якщо кількість послідовних відхилень (зв'язків через two) перевищує 4 рівні, LLM зобов'язана примусово розбити алгоритм на "Силует", створивши нові паралельні гілки (branch).¹
- **Обсяг контенту:** Текст у полі content одного вузла не повинен перевищувати 8-10 рядків логіки. Якщо блок більший, його слід семантично згорнути або виділити у підпроцес (іконка insertion).

Блок G — Validator Contract

Блок валідації (Deterministic Validator Module) є критичним бар'єром між генерацією LLM та фронтендом. Він працює на бекенді або в середовищі агента (наприклад, як частина MCP-сервера) і виконує перевірки та автоматичні виправлення (Autofix).¹

1. Порядок перевірок (Validation Pipeline Sequence)

Процес валідації виконується у суворо визначеному порядку — від синтаксису до топології:

1. **JSON Syntax & Schema Validation:** Первинна перевірка отриманого payload за допомогою Zod (або еквівалентної бібліотеки) на відповідність JSON Schema з Розділу 6. Це гарантує наявність ключа items та правильність переліків типів ікон (Enums).¹
2. **Topological Integrity Check (Dangling Pointers):** Валідатор проходить по всіх

об'єктах у items і збирає всі значення полів one, two, side. Далі він перевіряє, чи існують ці значення як ключі в items.

- *Autofix*: Якщо знайдено вказівник на неіснуючий вузол, валідатор програмно генерує пустий вузол типу action з текстом `/* Auto-generated stub */` для збереження планарності графа.

3. **Terminal Node Convergence (Правило одного End)**: Валідатор виявляє всі вузли, з яких потік не має продовження (відсутнє поле one).

- *Autofix*: Якщо таких вузлів більше одного, система залишає лише один вузол type: "end". Всі інші вузли змінюють тип на action (або address для Силуету), а в їхнє поле one записується ID збереженого фінального вузла.¹

4. **Orphan Trimming (Пошук сиріт)**: Запуск алгоритму Depth-First Search (DFS) від вузла header.

- *Autofix*: Всі вузли словника items, які не були відвідані під час обходу, вважаються "сиротами" (dead code) і автоматично видаляються з JSON.¹

5. **Twin Error Detection (Семантика Силуету)**: Перевірка перехресних посилань між гілками Силуету. Забороняється пряме підключення one/two між вузлами різних гілок (Twin error).⁴

- *Human Review*: Якщо виявлено пряме з'єднання гілок, система повертає помилку розробнику, оскільки автоматичне виправлення може порушити логіку скінченного автомата.

2. Формат повідомлень про помилки (Error Protocol)

Якщо помилка не піддається Autofix, або під час інтерактивного редагування, валідатор повертає структурований звіт про помилку через JSON-RPC ¹:

JSON

```
{
  "error_code": "ERR_TOPOLOGY_04_TWIN_ERROR",
  "severity": "human-review",
  "location": "items.node_auth_check.two",
  "message": "Direct link between Silhouette branches detected (node_auth_check -> branch_database).",
  "suggestion": "Replace the direct link with an 'address' node pointing to the target branch header."
}
```



Блок Н — Diagram Quality Checklist

Перед остаточним збереженням схеми в базу даних (наприклад, DrakonHub) або її відображенням на фронтенді, автоматизована система та користувач повинні оцінити якість згенерованої моделі. Цей чекліст базується на формалізованих метриках читабельності Володимира Паронджанова та сучасних евристичних мережевого аналізу.¹

1. Автоматизовані метрики читабельності (Automated Metrics)

Система обчислює наступні індекси на льоту:

- **Skewer Integrity Score (SIS - Індекс цілісності шампура):** Обчислюється як відношення кількості вузлів, що лежать на головному вертикальному шляху (визначається проходом виключно по one від header до end), до загальної кількості вузлів алгоритму.
 - *Оптимум:* > 40%. Якщо показник падає нижче 20%, алгоритм є надмірно розгалуженим і його слід декомпонувати на Силует.
- **Rightward Degradation Count (RDC - Глибина деградації вправо):** Вимірює максимальну глибину вкладеності ланцюгів переходу через атрибут two.
 - *Оптимум:* Не більше 3-4 рівнів. При значеннях > 5 виникає значне когнітивне перевантаження читача, схема маркується як така, що потребує Human Review.
- **Logical Density (Щільність логіки):** Середня кількість рядків коду всередині поля content вузлів. Якщо показник високий, це свідчить про те, що LLM згорнула забагато логіки, порушуючи принцип наочності.

2. Human Review Checklist (Чекліст для розробника)

- [] **Семантика Happy Path:** Чи відповідає головний шампур (пряма вертикаль) справжньому бізнес-очікуванню успішного сценарію?
- [] **Захист Спільної Долі (Common Fate):** Чи зведені однотипні обробники помилок (наприклад, повернення HTTP 500) до єдиного блоку на крайній правій осі, замість дублювання контенту?
- [] **Оптимізація умовних формул:** Чи розбиті складні текстові умови (на зразок if (A and B or C)) на окремі візуальні ікони Question у формі "драбинки"? Це є ключовою Best Practice для DRAKON.¹
- [] **Модульність:** Чи використано ікону insertion для виклику зовнішніх процесів, які не є критичними для розуміння даного графа?

Executive Summary

У цій інженерній специфікації детально описано архітектуру AI-системи, що генерує діаграми за стандартом візуальної алгоритмічної мови DRAKON.

Специфіковано:

1. **Гібридна конвеєрна архітектура**, яка делегує семантичний аналіз LLM, а геометричний рендеринг — детермінованим математичним алгоритмам, щоб уникнути "просторових галюцинацій".
2. **Доменна модель та формат IR**, що базується на плоскому JSON-словнику списку суміжності (adjacency list), оптимізованому для AI-генерації без глибокої вкладеності AST.
3. **Таксономія іконок та каталог правил**, які переводять когнітивні принципи DRAKON (заборона перетинів ліній, утримання вертикального шампура, евристика правої деградації) у жорсткі програмні обмеження (Hard/Soft constraints).
4. **Контракти LLM та Валідатора**, які встановлюють порядок обробки даних, автоматичного виправлення (Autofix) структурних порушень (сироти, обірвані вказівники) та метрики оцінки читабельності діаграм.

Implementation Readiness

До імплементації повністю готові:

- **Специфікація формату даних (Canonical IR):** JSON Schema є валідною і може бути безпосередньо імplementована як `response_format` у GPT-4 API або як валідаційний шар у MCP-сервері.
- **Validation Pipeline:** Алгоритми обходу графа (Depth-First Search) для пошуку сиріт та зведення кінцевих вузлів (Terminal Node Convergence) формалізовані та готові до написання на Node.js / Rust.
- **LLM Instruction Contract:** Матриця інструкцій та заборон повністю сформована для інтеграції у System Prompt AI-агента.

Unresolved Questions

Питання, які вимагають додаткового технічного дослідження перед стартом бекенд-розробки:

1. **Макроіконки (For Loop & Concurrent):** Оскільки `drakonwidget.js` обробляє типи `foreach` специфічно, залишається відкритим питання: чи повинна LLM генерувати два окремі вузли (Start і End) і самостійно замикати цикл через `one/two`, чи віджет здатен автоматично розгортати один логічний вузол циклу? Необхідний аудит функції `buildIconCore` у вихідному коді віджета.³
2. **Реактивність редагування (State Management):** У межах гібридної архітектури віджет використовує об'єкт `EditSender` (Vanilla JS) для синхронізації стану. Як оптимально адаптувати ці імперативні "дельти" в декларативний потік Zustand/Redux, щоб операції Autofix від MCP-сервера миттєво відмальовувалися без "блмання" Canvas?¹
3. **Resolving Dynamic Types:** Для мов на кшталт Python, наскільки глибоко парсер

(code2flow) має імплементувати евристичний пошук зв'язків між модулями, перш ніж передавати граф у LLM для семантичної оцінки?.¹

Handoff to Prompt 5

Для наступного етапу проєктування архітектури (Prompt 5) передаються такі вхідні дані:

- **JSON Schema (Draft) з Розділу 6** — виступатиме як Data Transfer Object (DTO) контракт для розробки бекенду.
- **Validation Pipeline (Розділ 8)** — технічне завдання для імплементції мікросервісу Graph Validator з алгоритмами Autofix.
- **Contract Matrix (Розділ 7)** — фундамент для налаштування Agentic Flow та System Prompts у LLM-оркестраторі (наприклад, LangChain або Anthropic MCP).
- **Diagram Quality Checklist (Розділ 9)** — база для впровадження AI Critic моделі або CI/CD літера діаграм.

Джерела

1. Дослідження Code-to-Diagram для DRAKON.pdf
2. DRAKON - Wikipedia, доступ отримано травня 9, 2026, <https://en.wikipedia.org/wiki/DRAKON>
3. stepan-mitkin/dragonwidget: A JavaScript widget for viewing and editing drakon flowcharts - GitHub, доступ отримано травня 9, 2026, <https://github.com/stepan-mitkin/dragonwidget>
4. DRAKON-Erlang part 6 - DRAKON Editor, доступ отримано травня 9, 2026, <https://drakon-editor.sourceforge.net/drakon-erlang/silh.html>
5. DRAKON.pdf, доступ отримано травня 9, 2026, <https://drakon-editor.sourceforge.net/DRAKON.pdf>
6. DRAKON visual programming language — communication tool for ..., доступ отримано травня 9, 2026, <https://www.mql5.com/en/articles/13324>
7. DRAKON-AutoHotkey: Visual programming with AutoHotkey - AutoHotkey Community, доступ отримано травня 9, 2026, <https://www.autohotkey.com/boards/viewtopic.php?t=3108>
8. DRAKON Language Reference - DrakonFlow, доступ отримано травня 9, 2026, <https://drakonflow.com/read/drakon-reference>
9. A Quantitative Insight into the Impact of Translation on Readability - ACL Anthology, доступ отримано травня 9, 2026, <https://aclanthology.org/W14-1212.pdf>